

Инвариантное задание 1.1. Подготовить обзор программного продукта для разработки.

1. Общая характеристика

PyCharm – это интегрированная среда разработки (IDE) для языка Python, разработанная компанией JetBrains. Это один из самых популярных и мощных инструментов для профессиональной разработки на Python, который поддерживает разработчика на всех этапах: от написания кода до тестирования и отладки.

Ключевые особенности:

- Кроссплатформенность – работает на Windows, macOS и Linux
- Умное редактирование кода с автодополнением и анализом на лету
- Встроенные инструменты: отладчик, тестирование, система контроля версий (Git)
- Поддержка веб-фреймворков (Django, Flask) и научных библиотек
- Версии PyCharm:
 - Community (бесплатная) – подходит для изучения Python и чистого программирования
 - Professional (платная) – включает поддержку веб-разработки, работу с базами данных и научные инструменты

Начинающему программисту обычно достаточно Community Edition – она бесплатна и содержит все необходимые функции для учебных проектов.

2. Необходимое программное и аппаратное обеспечение (requirements)

2.1 Аппаратные требования

Таблица 1 – Системные требования PyCharm

Компонент	Минимальные	Рекомендуемые
ОЗУ	8 ГБ (из них 3 ГБ свободно для IDE)	16 ГБ
Место на диске	10 ГБ	20+ ГБ на SSD
Процессор	4 ядра, x86_64 или arm64	6–8 ядер, x86_64 или arm64
Монитор	1280×720	1920×1080

2.2 Программные требования

- Операционная система: Windows 10/11 (64-bit), macOS 12.0+, Linux (Ubuntu LTS, Fedora)
- Python: версии 2.7 или 3.9-3.15 (среда поддерживает оба поколения)
- Java: устанавливать отдельно не требуется – PyCharm включает встроенную среду JetBrains Runtime (JBR 21)

3. Функции PyCharm

3.1 Создание проекта

При запуске PyCharm предлагает создать новый проект. В диалоговом окне необходимо указать:

- Расположение проекта на диске
- Тип проекта (Pure Python, Django, Flask и др.)
- Интерпретатор Python – рекомендуется создать виртуальное окружение (virtualenv), чтобы изолировать зависимости проекта

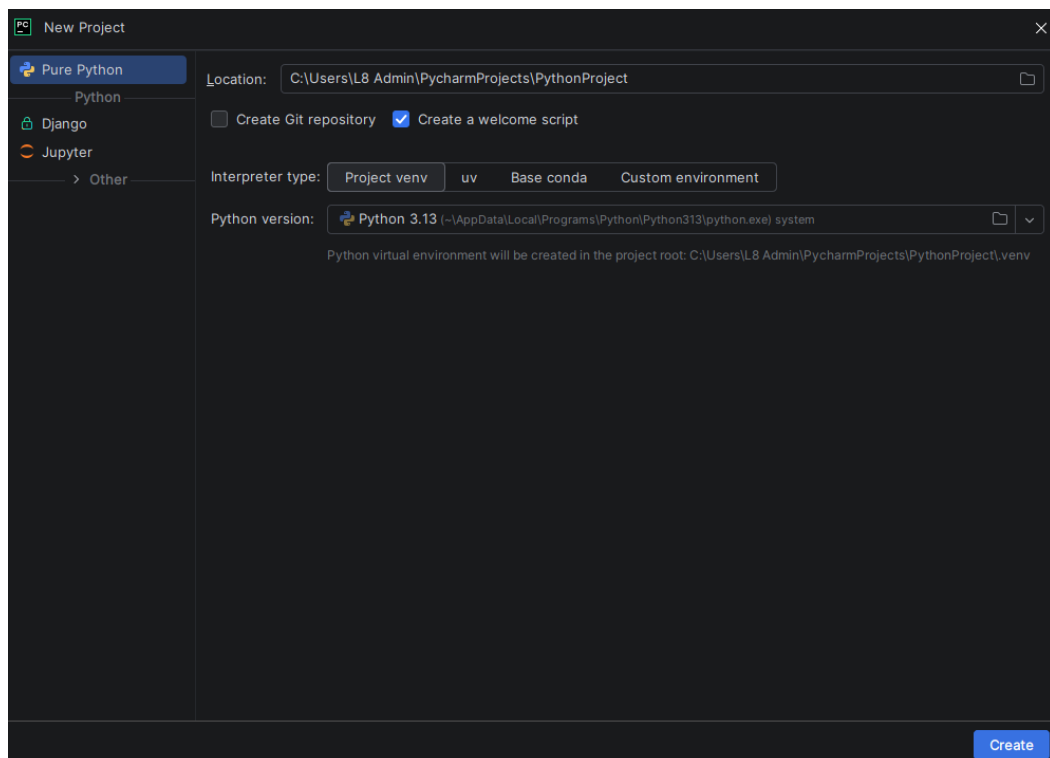


Рисунок 1 – Диалог создания нового проекта (New Project) с выбором расположения и интерпретатора

3.2 Кодирование (написание кода)

PyCharm предоставляет интеллектуальные инструменты для написания кода:

- Автодополнение (Ctrl+Space) – подсказывает переменные, функции, классы с учётом контекста
- Умное завершение (Ctrl+Shift+Space) – фильтрует методы по типу переменной
- Автоматическая расстановка отступов – отступы добавляются автоматически в начале новой строки
- Подсветка ошибок на лету – ошибки отображаются прямо при наборе кода
- Быстрые исправления (Alt+Enter) – предлагает варианты исправления проблем

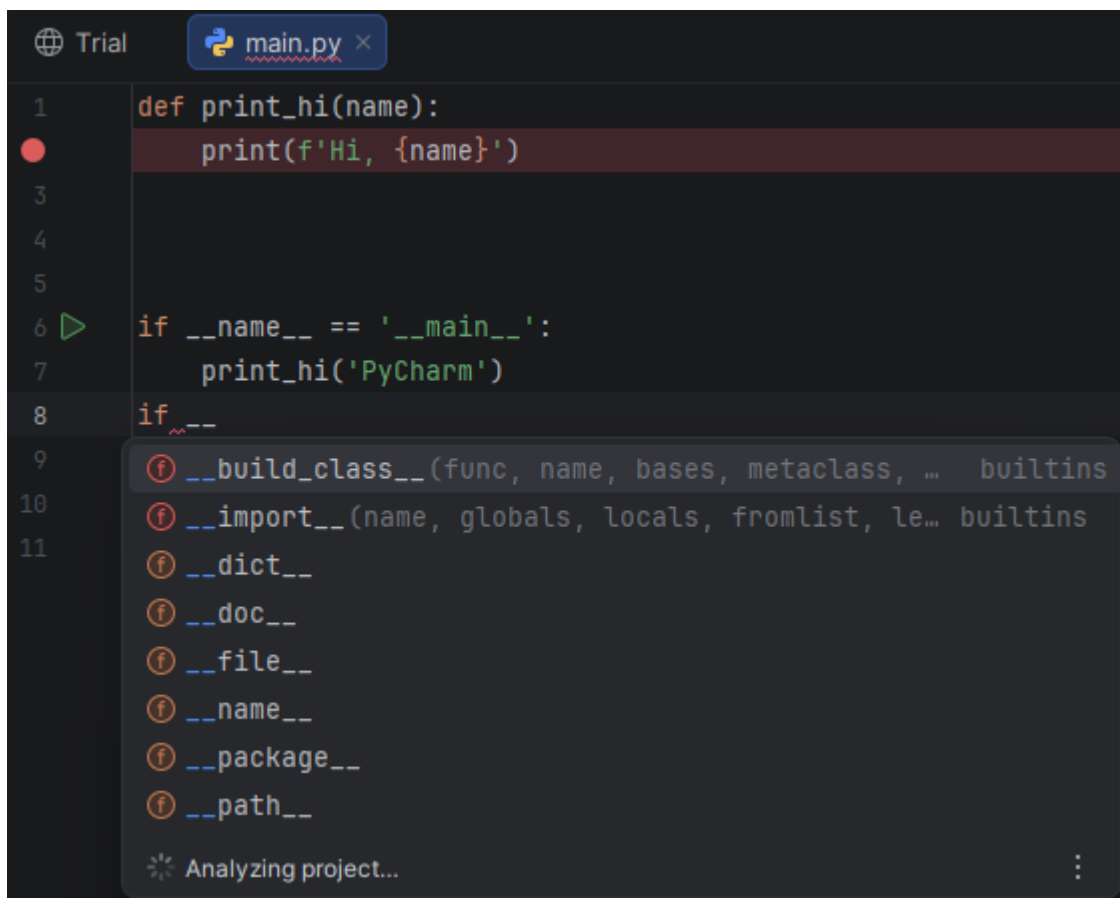


Рисунок 2 – Контекстное автодополнение кода в редакторе

3.3 Форматирование кода

PyCharm включает встроенный формater, соответствующий стандарту **PEP-8** для Python:

- Автоформатирование (Ctrl+Alt+L) – приводит код к единому стилю
- Настраиваемый стиль кода – можно выбрать один из готовых стилей для каждого языка
- Анализ кода на соответствие PEP-8 – IDE предлагает исправления через Alt+Enter



The screenshot shows a PyCharm code editor with a dark theme. The file name is 'main.py'. The code is as follows:

```
1 import time
2
3 def Calc( L ):
4     res = []
5     for i in range(len(L)):
6         if L[i] > 0:
7             res.append(L[i] * 2)
8         else:
9             res.append(L[i])
10    return res
11
12 x = [-2, 5, 0, 3, -1]
13 print(Calc(x))
14
```

Рисунок 3 – Фрагмент кода до автоформатирования



The screenshot shows the same PyCharm code editor after applying the auto-formatting (Ctrl+Alt+L). The code is now formatted according to PEP-8 standards:

```
1 import time
2
3 def Calc( L : {__len__, __getitem__} ): 1 usage
4     res = []
5     for i in range(len(L)):
6         if L[i] > 0:
7             res.append(L[i] * 2)
8         else:
9             res.append(L[i])
10    return res
11
12 x = [-2, 5, 0, 3, -1]
13 print(Calc(x))
```

Рисунок 4 – Тот же фрагмент после автоформатирования (Ctrl+Alt+L)

3.4 Отладка (Debugging)

Отладчик PyCharm позволяет найти причины ошибок во время выполнения программы:

- Точки останова (breakpoints) – устанавливаются щелчком на левом поле редактора (gutter). Программа останавливается в этом месте, позволяя изучить состояние переменных
- Запуск отладки (Shift+F9) – запускает программу в режиме отладки
- Пошаговое выполнение:
 - Step Over (F8) – выполнить строку, не заходя внутрь функций
 - Step Into (F7) – зайти внутрь вызываемой функции
 - Step Out (Shift+F8) – выйти из текущей функции
- Панель Debugger – отображает значения переменных в реальном времени

4. Запуск (Run)

PyCharm запускает Python-скрипты и проекты через встроенные конфигурации запуска (Run/Debug Configurations) – именованные наборы параметров, определяющих, что и с какими настройками выполнять.

- Запуск в один клик: правый клик по файлу → «Run» или сочетание клавиш **Shift+F10**.
- Готовые шаблоны конфигураций: обычный Python-скрипт, *Python tests* (pytest/unittest), Django, Flask, Jupyter Notebook, запуск через удалённый интерпретатор (SSH, WSL, Docker).
- Конфигурации бывают временными (создаются автоматически) и постоянными (сохраняются в проекте, файл в директории *.idea/*).
- Вывод программы отображается во встроенной консоли (Run tool window); туда же можно передавать ввод (stdin).
- Поддерживается *Attach to Process* – подключение к уже запущенному Python-процессу.

- Режим *Run with Coverage* – запуск с одновременным подсчётом покрытия кода тестами; результат подсвечивается прямо в редакторе (зелёным/красным по строкам).
- Составные (compound) конфигурации – запуск нескольких конфигураций одновременно одной кнопкой (например, backend и frontend вместе).

Подробнее: jetbrains.com/help/pycharm/run-debug-configurations.html

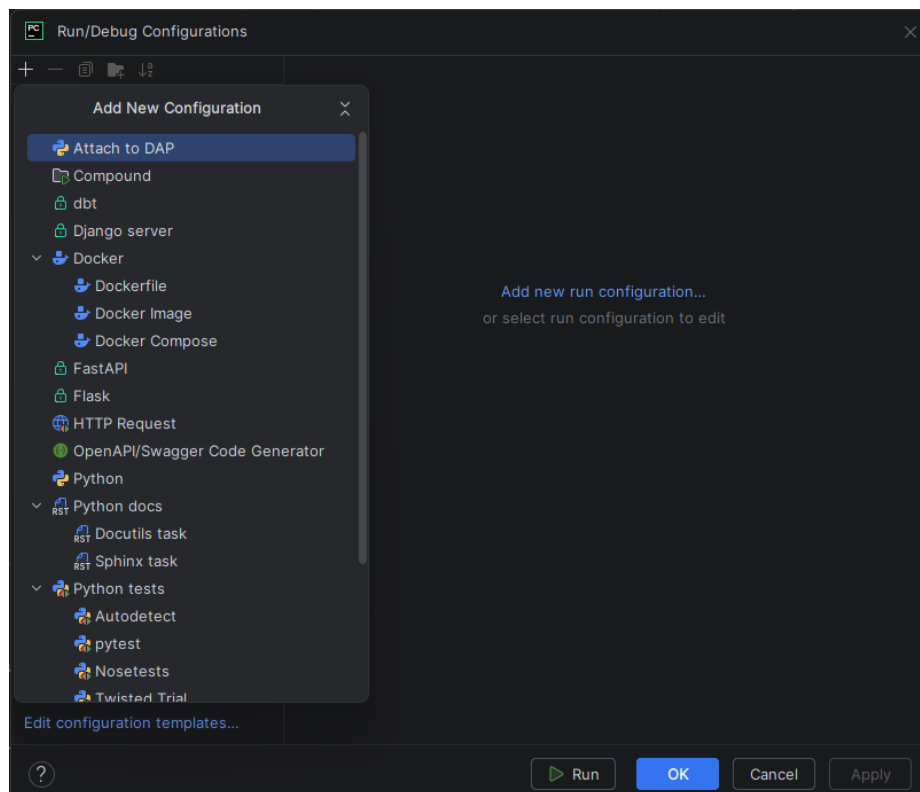


Рисунок 5 – Окно редактора конфигураций запуска (Edit Configurations)

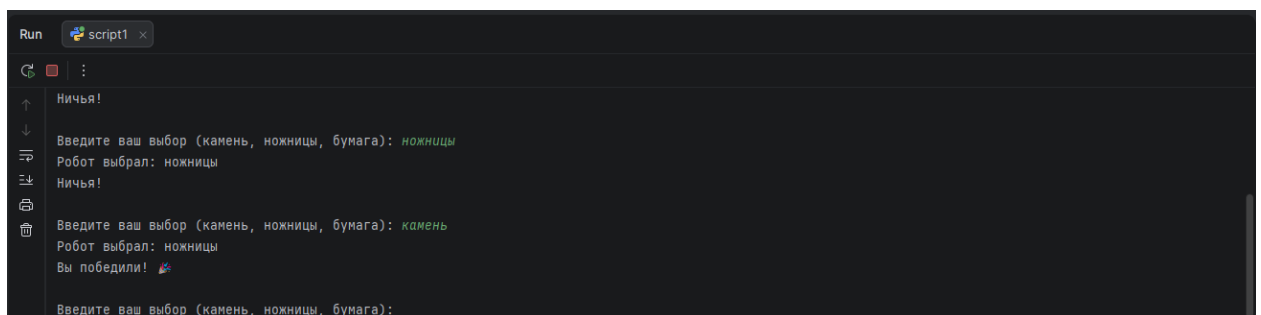


Рисунок 6 – Консоль Run с выводом выполненной программы

5. Компиляция

Python – интерпретируемый язык, поэтому классической компиляции в исполняемый файл (как в C++/Java) в PyCharm нет. Тем не менее с этим процессом связаны следующие возможности IDE:

- Автоматическое создание байт-кода *.pyc* при запуске – этим занимается сам интерпретатор Python, а PyCharm использует результат для индексации и ускорения последующих запусков.
- Поддержка сборки расширений на C/Cython через настроенные конфигурации (*setup.py build_ext* и аналогичные сценарии).
- Управление зависимостями перед запуском («сборка окружения»): встроенный инструмент *Python Packages* – установка пакетов через *pip*, *uv*, *Poetry* или *conda* прямо из интерфейса.
- Создание и переключение виртуальных окружений (*venv*, *conda*) прямо при создании проекта или позже, через *Settings* → *Python Interpreter*.
- Если версия пакета в *requirements.txt/pyproject.toml* не совпадает с установленной – PyCharm подсвечивает это и предлагает quick-fix «Install requirement».
- Начиная с версии 2026.1-2026.2 – расширенная поддержка *uv* как менеджера окружений и работа с *multi-project/workspace*-конфигурациями.



Рисунок 7 – Окно Python Packages со списком установленных зависимостей проекта

6. Версионирование (VCS)

PyCharm имеет встроенную интеграцию с системами контроля версий – в первую очередь с Git (также поддерживаются Mercurial, SVN, Perforce).

- Меню **VCS / Git** и панель **Commit**: инициализация репозитория, добавление файлов в индекс (stage), коммит, push, pull, fetch.
- Создание и переключение веток, merge, rebase; разрешение конфликтов через встроенный трёхпанельный *Merge Tool*.
- Вкладка *Git Log* – визуализация истории коммитов и графа веток.
- Функция *Annotate (Blame)* – построчный показ автора и времени последнего изменения строки кода.
- Локальная история изменений (*Local History*) – PyCharm сам сохраняет снимки файлов и позволяет откатиться даже без Git.
- Создание и редактирование *.gitignore* с подсказками прямо в IDE.
- *Shelve / Unshelve* – временное «откладывание» незакоммиченных изменений без создания коммита (встроенный аналог *git stash*).
- Настройка шаблонов commit-сообщений и commit-хуков (например, автоматический запуск тестов/линтера перед коммитом).

Подробнее: jetbrains.com/help/pycharm/version-control-integration.html

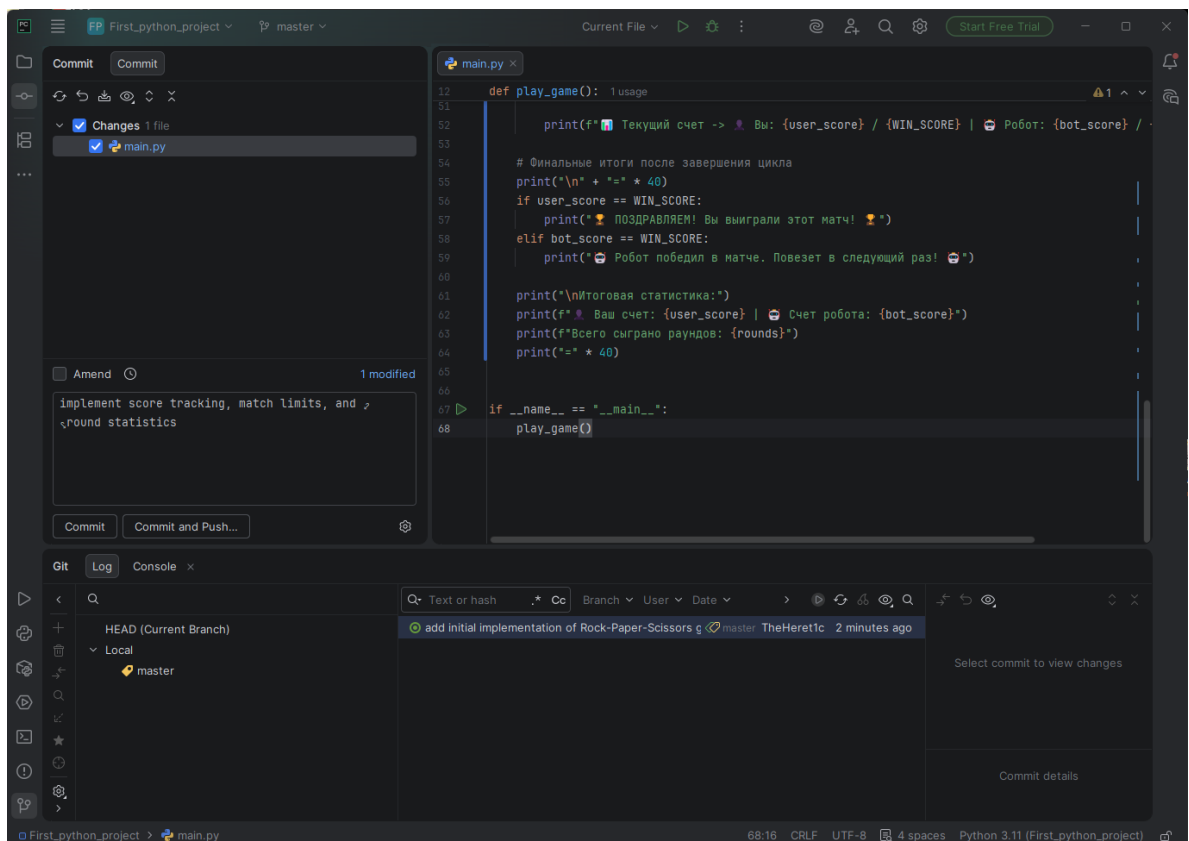


Рисунок 8 – Панель Commit со списком изменённых файлов и вкладка Git Log с историей КОММИТОВ

7. Публикация в репозитории

PyCharm позволяет публиковать и синхронизировать проект с удалённым репозиторием, не выходя из IDE.

- Прямая интеграция с GitHub (а также GitLab/Bitbucket через плагины): авторизация по токену/OAuth и создание нового удалённого репозитория командой *VCS* → *Share Project on GitHub*.

- Отправка изменений (push) одной кнопкой или сочетанием **Ctrl+Shift+K**.

- Работа с Pull Request'ами: создание, просмотр, ревью кода, комментирование – во встроенной вкладке *Pull Requests*.

- Просмотр статусов CI/CD (GitHub Actions) прямо в интерфейсе IDE.

- Готовые Python-шаблоны *.gitignore* при создании репозитория.

- Подключение нескольких remote-репозиториях одновременно и переключение между ними.

Подробнее: jetbrains.com/help/pycharm/manage-projects-hosted-on-github.html

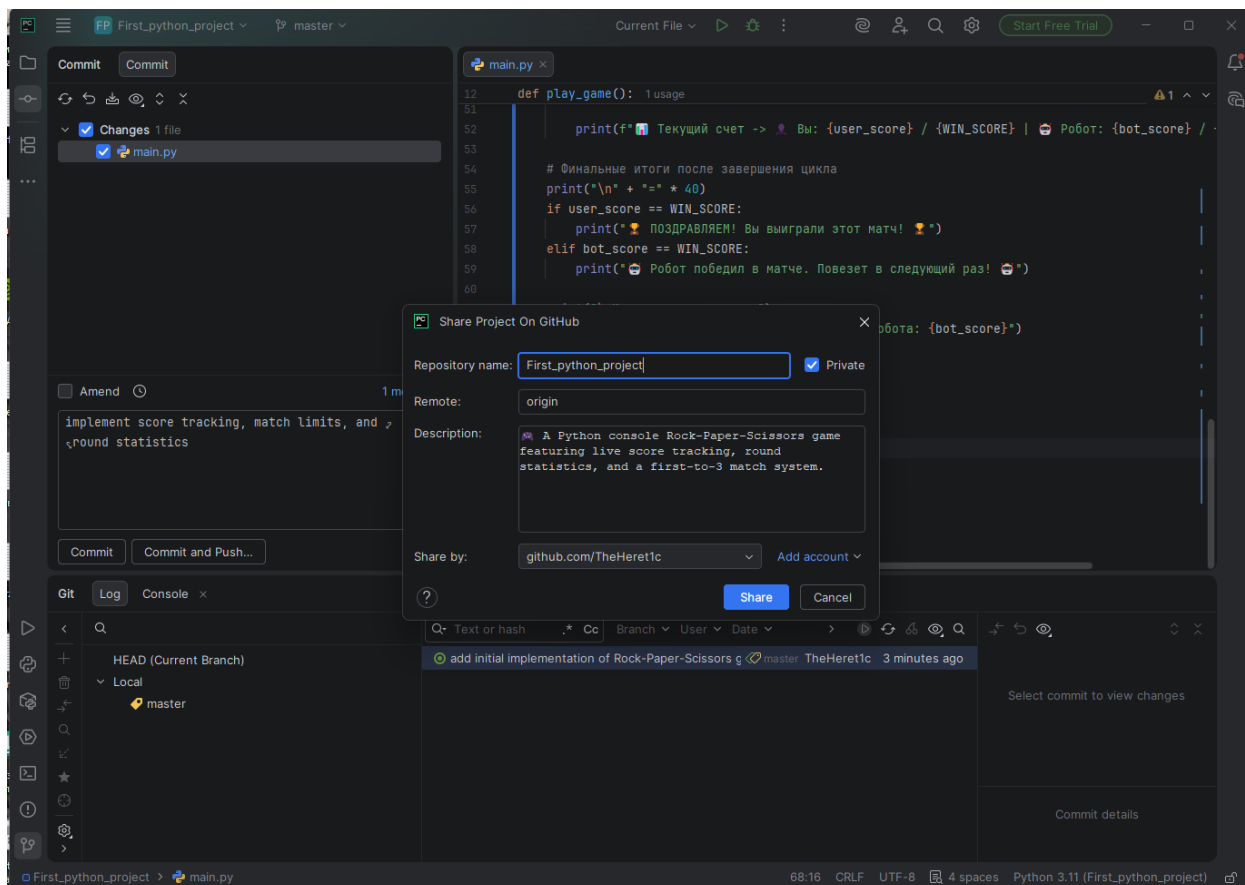


Рисунок 9 – Диалог «Share Project on GitHub» для публикации проекта в новый репозиторий на GitHub

8. AI-функции

В PyCharm встроен JetBrains AI Assistant, расширенный в последних версиях до полноценного AI-агента.

- AI-чат: вопросы по коду, документации, объяснение ошибок в контексте открытого проекта.
- AI-автодополнение кода: полнострочные и многострочные предложения, а не только подстановка по ключевым словам.
- Генерация кода по текстовому описанию (natural language → код), генерация docstring/комментариев и юнит-тестов.
- Объяснение и рефакторинг выделенного участка кода по запросу.
- AI-объяснение ошибок при сбое программы или в отладчике.
- Автоматическая генерация текста commit-сообщения по diff изменений – кнопка рядом с полем сообщения коммита.

- Поддержка сторонних AI-агентов через протокол ACP (Agent Client Protocol) – Claude Agent, GitHub Copilot, Codex, Cursor и др., выбираемых прямо в AI-чате (начиная с версии 2026.1).
- Jupyter notebook skill для AI-агентов – агент может создавать, редактировать и выполнять ячейки ноутбука через живое ядро, сохраняя состояние переменных между вызовами.
- Agent environment coordinator – агент автоматически определяет, какой интерпретатор/менеджер окружения настроен в проекте (uv, Poetry, venv, conda), и устанавливает пакеты в нужное окружение.

Подробнее: jetbrains.com/ai-ides

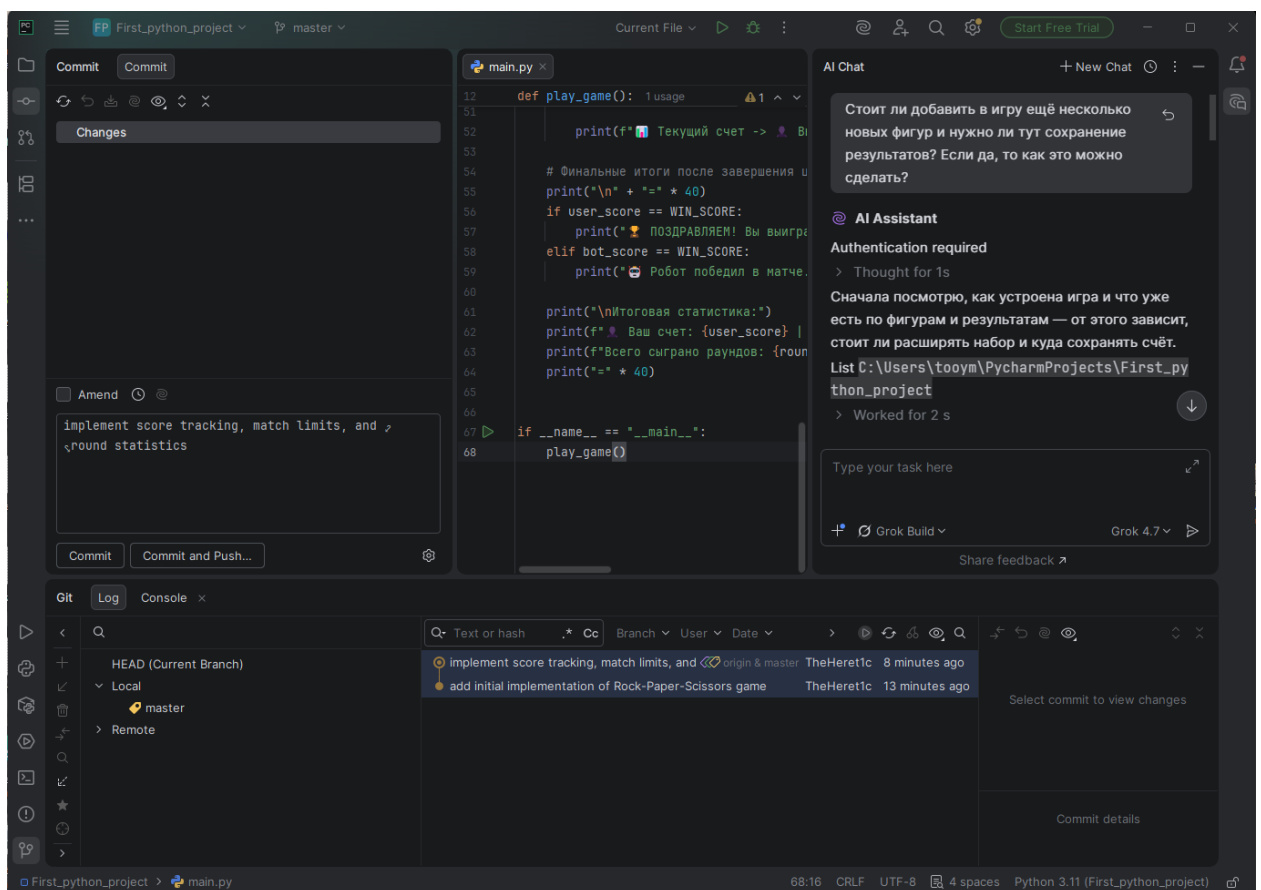


Рисунок 10 – Окно AI Assistant с чатом по коду открытого проекта

9. Дополнительные возможности среды

Помимо основных функций, в PyCharm есть ряд возможностей, которые заметно ускоряют разработку и часто упоминаются в обзорах IDE как её сильные стороны:

9.1 Работа с базами данных

- Встроенный инструмент *Database Tool Window* – подключение к БД (PostgreSQL, MySQL, SQLite и др.), просмотр и редактирование таблиц, выполнение SQL-запросов прямо из IDE без сторонних клиентов.

9.2 Встроенный HTTP-клиент

- Файлы *.http* позволяют писать и выполнять HTTP-запросы к своему API (полезно при разработке backend'a) прямо в редакторе, с сохранением истории запросов.

9.3 Docker и удалённая разработка

- Интеграция с Docker и Docker Compose – запуск, остановка контейнеров, просмотр логов из панели Services.
- PyCharm Remote Development / Gateway – можно открыть проект и работать с ним, физически находясь на удалённой машине или в контейнере, при этом интерфейс IDE остаётся локальным.

9.4 Рефакторинг и анализ кода

- Умные рефакторинги (Rename, Extract Method/Variable, Move) – применяются во всём проекте с автоматическим обновлением всех связанных мест.
- Structural Search and Replace – поиск и замена не по тексту, а по структуре кода (например, «найти все вызовы функции с определённым паттерном аргументов»).
- Постоянные инспекции кода (Code Inspections) – подсветка потенциальных ошибок, неиспользуемого кода, нарушений PEP 8 с быстрыми исправлениями.

9.5 Прочее

- Live Templates – быстрые сокращения для вставки шаблонов кода (например, *main* + Tab → заготовка `if __name__ == '__main__':`).

- Scientific Mode – режим для работы с данными: интерактивный просмотр DataFrame, построение графиков рядом с кодом (полезно при анализе данных на Python).
- Плагины и маркетплейс JetBrains Marketplace – расширение функциональности (поддержка дополнительных фреймворков, тем оформления, линтеров и т. д.).
- Встроенный терминал с несколькими вкладками, работающий прямо в контексте проекта и активного интерпретатора.

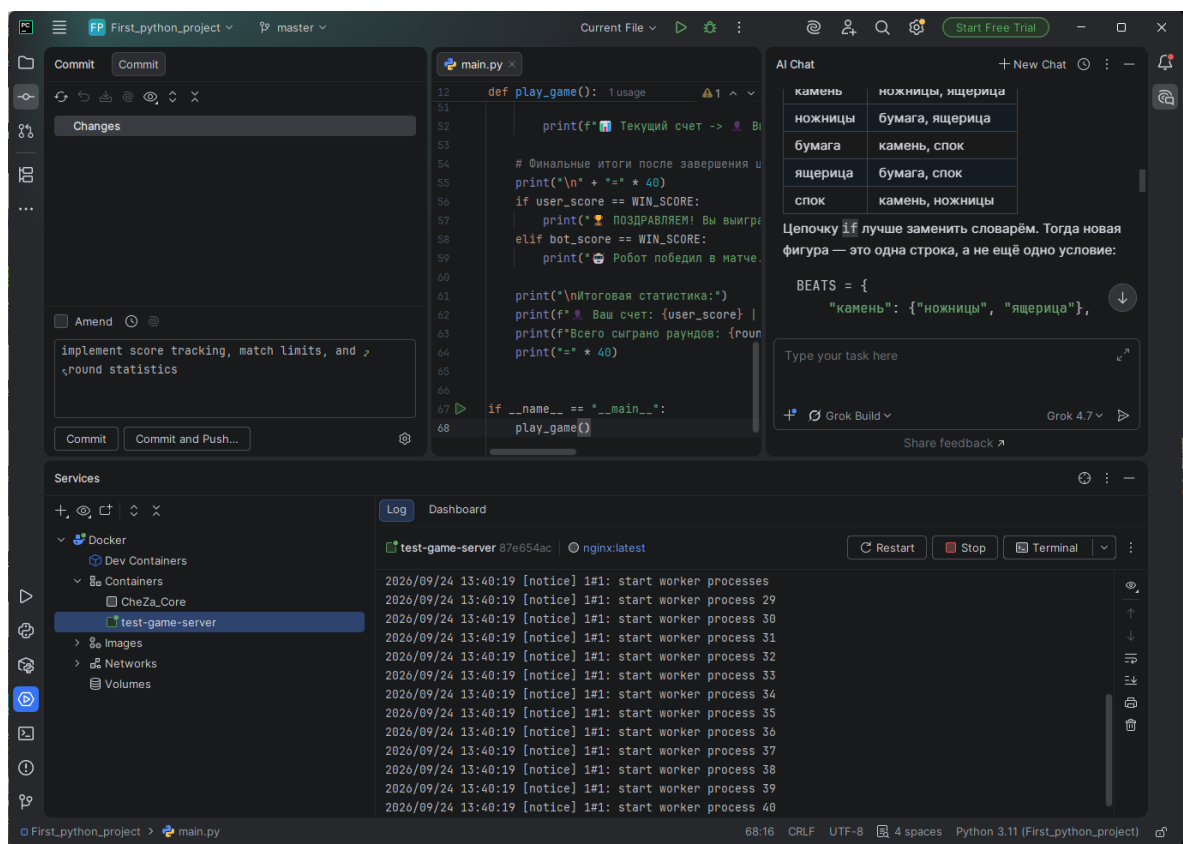


Рисунок 11 – Панель Services с запущенным Docker-контейнером проекта и его логами

Список источников

1. PyCharm Help – Run/debug configurations:
<https://www.jetbrains.com/help/pycharm/run-debug-configurations.html>
2. PyCharm Help – Version control integration:
<https://www.jetbrains.com/help/pycharm/version-control-integration.html>
3. PyCharm Help – Manage projects hosted on GitHub:
<https://www.jetbrains.com/help/pycharm/manage-projects-hosted-on-github.html>
4. JetBrains AI in PyCharm: <https://www.jetbrains.com/ai-ides/>
5. PyCharm Help – Database tool window:
<https://www.jetbrains.com/help/pycharm/relational-databases.html>
6. PyCharm What's New (релизы 2026.1-2026.2):
<https://www.jetbrains.com/pycharm/whatsnew/>
7. Рогачев А. Ф., Мелихова Е. В., Плещенко Т. В. «Установка и использование интегрированной среды PyCharm для разработки приложения на языке Python» – учебно-методическое пособие, Волгоградский ГАУ, 2024.